

2.1.2

2026-02-13

- **Whisper STT 모델 경량화:** 음성인식 Fallback 모델로 사용되는 Whisper의 모델 크기를 하향 조정함. 기본 모델을 `medium` 에서 `base` 로, 보조 Fallback 모델을 `small` 에서 `tiny` 로 변경하여 약 1.5GB 수준의 메모리를 절감함.
- **Elasticsearch 힙 메모리 축소:** 검색 엔진의 JVM 힙 할당을 512MB에서 **256MB**로 줄여 메모리 사용량을 최적화함. Lightsail 2GB RAM 환경 기준으로 **ES 256MB + Qdrant ~200MB + Redis 50MB + API 서버 512MB + 프론트엔드 200MB + Whisper(base) 150MB ≈ 총 1.4GB** 수준으로 메모리 사용을 안정화함.

2.1.1

2026-02-11

- **ML 재랭킹 모드 확장:** 외부 LLM을 호출하지 않고도 동작하는 `simulated` 모드와 `local` 모드를 ML 재랭킹 서비스에 추가함. `simulated` 모드는 환경변수 기반으로 설정된 가상 지연 시간 (`SIM_TARGET_LATENCY_MS` 등)과 에러율(`SIM_TIMEOUT_RATE`, `SIM_RATE_LIMIT_RATE` 등)을 적용하여 응답을 시뮬레이션하고, 오류 발생 시 로컬 키워드 기반 후보 선택으로 **fallback** 처리함. `local` 모드는 토큰 겹침 점수 산정으로 항상 동일한 결과를 주는 **결정적** 재랭킹을 수행하며, 산출된 신뢰도를 0~1 범위로 정규화함.
- **외부 LLM 호출 비용 제어:** 재랭킹을 실제 외부 LLM(Vendor)로 수행하는 빈도를 조절하기 위해 `VENDOR_SAMPLE_RATE` 환경변수를 추가함. `VENDOR_SAMPLE_RATE` 값을 0.0으로 설정하면 재랭킹 시 **항상 로컬 대안**을 사용하고 외부 LLM 호출을 차단하며, 1.0이면 모든 재랭킹 요청에 LLM을 호출함(기본값 1.0).
- **에러 응답 표준화 및 기본값 변경:** 재랭킹 결과의 오류 유형을 `RATE_LIMIT`, `TIMEOUT`, `VENDOR_ERROR` 등의 값으로 표준화하여 `error_type` 필드로 제공함. 외부 벤더 LLM 호출 실패 시 예외 메시지 대신 `error_type="VENDOR_ERROR"`로 명시하고, **기본 재랭킹 모드**를 종전 `mock` 에서 `simulated` 모드로 변경함.

2.1.0

2026-02-10

- **ML 재랭크 서비스 도입:** 검색 결과 후처리 단계를 위해 **ML 재랭킹 서비스**를 추가함. `RerankService` 클래스에서 환경변수 `RERANK_MODE` (`mock` 혹은 `vendor`)에 따라 두 가지 방식으로 동작함. `mock` 모드에서는 첫 번째 후보를 선택하는 모의 로직으로 거의 **0ms** 지연을 보이고, `vendor` 모드에서는 기존 Gemini LLM 기반 재랭킹 함수(`backend.logic.reranker.rerank_candidates`)를 호출하여 **AI 모델로 최적 후보를 선정**함. 공통으로 재랭크 결과는 `selected_id`, `reason`, `confidence`, `latency_ms`, `is_fallback`, `error_type` 등의 **표준화된 필드 구조**로 반환하며, LLM 호출 실패 시 `is_fallback=true` 처리 및 `error_type`에 오류 원인을 명시하여 **안정성**을 높였음.
- **재랭크 전용 API 엔드포인트 추가:** 재랭킹 모듈의 부하 테스트와 모니터링을 위해 `POST /ml/rerank` REST API 엔드포인트를 신설함. 이 엔드포인트를 통해 별도의 입력(JSON 후보 목록)에 대해 재랭킹을 수행할 수 있으며, 응답 헤더 `X-Rerank-Latency-MS`로 실제 처리 소요 시간을 밀리초 단위로 제공하여 외부에서 **QPM**(분당 쿼리) 모니터링이 가능하도록 함.

- **부하 테스트 스크립트 추가:** 재랭킹 서비스의 성능과 안정성을 검증하기 위해 k6 기반의 스크립트(`scripts/loadtest_rerank.js`)와 Python 비동기 HTTP 부하 테스트 스크립트(`scripts/loadtest_rerank.py`)를 도입함. 다양한 시나리오에서 재랭킹 요청을 반복 실행하여 **평균 지연시간** 및 **p95/p99 지연시간**을 측정하고, 분당 처리 건수 등 성능 지표를 평가할 수 있도록 함.

2.0.0

2026-02-10

- **모호성 판별 모듈 추가:** 사용자 질문에 대한 검색 결과가 애매하거나 다의적인 경우를 처리하기 위해 **모호성 탐지** 기능을 도입함. `backend/logic/ambiguity.py` 모듈에서 검색 결과를 분석하여 `AmbiguityType` (없음 `NONE`, 카테고리 범위 넓음 `BROAD_CATEGORY`, 설명 모호 `VAGUE_DESCRIPTION`, 다중 의도 `MULTI_INTENT`, 결과 없음 `NO_RESULTS`)을 판정하고, 주요 **카테고리 분포**를 계산하여 **후속 Clarification 질문 옵션**을 생성함. 또한 동일 질문에 대해 최대 2회까지만 추가 질문을 하고 그 후에도 모호하면 **자동 키워드 검색으로 대체하는 2-회 제한(fallback)** 로직을 구현함.
- **LLM 재랭킹 고도화:** 검색 결과 후보 중 최적의 답변을 고르는 **재랭커 모듈**(`backend/logic/reranker.py`)을 개선함. Google **Gemini 2.0 Flash** LLM을 활용하여 각 후보의 적합도를 **추론(CoT)**하고 0.0~1.0의 **신뢰도 점수**와 선택 사유를 부여함. LLM을 사용할 수 없을 때는 키워드 매칭 기반의 재랭킹으로 **자동 폴백**하며, 어떤 후보도 적합하지 않은 경우 `selected_id`를 `null`로 반환하는 등 예외 상황에서도 안정적으로 동작하도록 함.
- **데이터 스키마 확장:** 모호성 및 재랭킹 정보 전달을 위해 데이터 모델을 확장함. 새로운 `AmbiguityType` 열거형을 정의하고, NLU 분석 결과 모델(`NLUResponse`)에 `ambiguity_type` 필드와 `confidence` 필드를 추가함 (기본값 각각 `NONE` 과 `1.0`).
- **검색 파이프라인 및 API 개선:** 통합 검색 파이프라인(`backend/logic/integrated_search.py`)에 **Step 4: 모호성 탐지 및 Clarification 질문 생성 단계**와 **Step 5: LLM 재랭킹 단계**를 추가 통합함. 이로써 검색 응답에 모호성 여부와 재랭킹 결과가 반영되며, API 응답 스키마(`SearchResponse`)에 `needs_clarification` (추가 질문 필요 여부), `clarification_question` (추가 질문 텍스트), `clarification_options` (선택지 목록), `clarification_count` (현재까지 진행된 Clarification 횟수), `is_fallback` (재랭킹 폴백 여부) 필드가 새로 추가됨. 또한 API 요청 모델(`SearchRequest`)에도 `clarification_count` 필드를 포함시켜 클라이언트가 현재까지 받은 Clarification 시도를 서버에 전달, 다회차 문맥을 유지할 수 있도록 함.

1.0.0

2026-02-10

- **하이브리드 검색 인프라 도입:** 대규모 벡터 검색을 지원하기 위해 **서버 B**(Vector Store/Data)에 **Elasticsearch 8.13.4**(BM25 텍스트 검색 엔진), **Qdrant v1.9.7**(벡터 DB), **Redis 7.2**(캐시)로 구성된 **하이브리드 검색 클러스터**를 구축함. Docker Compose 설정(`docker-compose.yml`)을 통해 서버 A(FastAPI 앱)와 분리된 검색 서비스 인프라를 추가하여 운영 환경(Lightsail 2GB)에서 **BM25 + 벡터** 복합 검색이 가능해짐.
- **검색 엔진 구조 개편:** 검색 정확도 향상을 위해 **벡터 임베딩 모델을 교체**하고 복합 검색 로직을 구현함. 기존 CLIP(512차원) 임베딩을 **Google Gemini 임베딩(3072차원)**으로 교체하여 텍스트 의미 벡터화를 고도화했고, Elasticsearch의 **BM25 점수**와 Qdrant의 **벡터 유사도 점수**를 **RRF 가중치 융합**하는 **하이브리드 검색 알고리즘**을 도입함. 이를 지원하는 **색인기**(`backend/search/indexer.py`)를 개발하여 상품 **카탈로그 데이터**를 Elasticsearch와 Qdrant에 동기화하고, **검색 품질 벤치마크 도구**(`backend/search/benchmark.py`)를 통해 하이브리드 검색의 성능을 검증함 (예: Hit@5 ~99%로 BM25 단일 검색 대비 정확도 향상).

- **통합 파이프라인 적용:** 백엔드 통합 검색 로직(`integrated_search`)을 신규 하이브리드 검색 구조로 연결함. 이에 따라 사용자의 질의에 대해 우선 외부 검색 인프라(ES+Qdrant)를 활용하고, 만약 외부 검색 서비스가 응답하지 않을 경우 **로컬 SQLite** 기반 검색으로 **자동 폴백**되도록 처리하여 **신뢰성**을 강화함.
 - **환경변수 구성 업데이트:** 새로운 검색 아키텍처에 맞추어 설정 파일 및 환경변수 구성을 확장함. `.env` 및 `.env.example` 에 `QDRANT_URL`, `ES_HOST`, `REDIS_HOST` 등 **외부 벡터/검색 서비스 접속 정보**와 임베딩 API 키 등이 추가되었고, 애플리케이션이 이러한 설정을 `backend/search/config.py` 등을 통해 불러와 사용하도록 함.
-